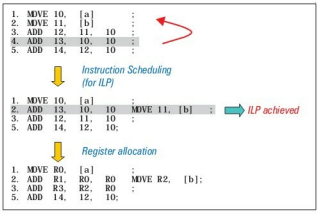

Figure 5: Method II used on code sample

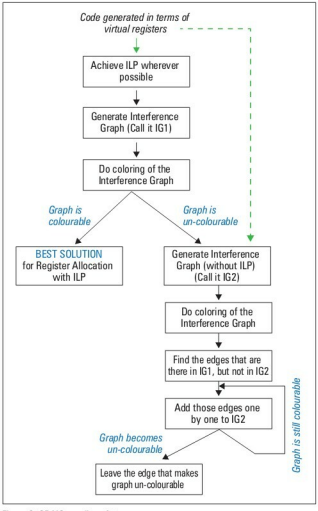

Figure 6: CRAIG—outline of steps

Method II: Register allocation after instruction scheduling

When register assignment is done after instruction scheduling, it is called late register assignment. Figure 4 depicts this process. Here, we try to allocate as many different registers as possible to prevent anti-dependencies so the scheduler can generate an efficient schedule. However, this often increases the amount of interference between program variables.

For instance, let's take the same instruction sequence as in the previous example. We apply Method II, first performing instruction scheduling for ILP, and then register allocation. Thus, we have the ILP and register allocation as shown in Figure 5.

In Figure 5, we have done late register assignment (after instruction scheduling). Now, we have four resultant instruction cycles, but four registers are used this time: *R0*, *R1*, *R2*, and *R3*. Though we have used one additional register, we have prevented the false dependencies. Moreover, we were able to achieve parallelism (before register allocation itself), and the total number of instruction cycles was reduced. Hence, this is an improved schedule compared to Method I.

However, this method can lead to spilling. Here, we have made use of four registers. If there were only three physical registers available, then spill code would be needed, and the resulting schedule would be longer due to delays caused by additional memory accesses.

Method III: Register allocation and instruction scheduling hand-in-hand

As we have seen, Method I made optimal use of available registers, but created unnecessary dependencies due to register re-use. As a result, the scope of ILP was reduced. In Method II, we were able to generate an efficient schedule without unnecessary dependencies, but the liberal use of registers to achieve ILP could raise the problem of register spilling. Method III performs register allocation and instruction scheduling together. This method aims at lowered register pressure and increased flexibility in instruction scheduling and is based on 'Combining Register Assignment Interference Graph (CRAIG)' (see Reference 3). An outline of the steps for the CRAIG method is shown in Figure 6. In this method, we need to construct an interference graph to perform the register allocation. To learn how to build an interference graph, refer to my article on 'Dissecting Register Allocation' in the June 2010 issue of *LINUX For You*.

Consider the same example we discussed in the last section.

```
MOVE  10, [a]    ;
MOVE  11, [b]    ;
ADD   12, 11, 10 ;
ADD   13, 10, 10 ;
ADD   14, 12, 10 ;
```

To this code, we apply ILP as follows:

```
MOVE  10, [a]    ;
MOVE  11, [b]    ADD  13, 10, 10  ;
ADD   12, 11, 10 ;
ADD   14, 12, 10 ;
```